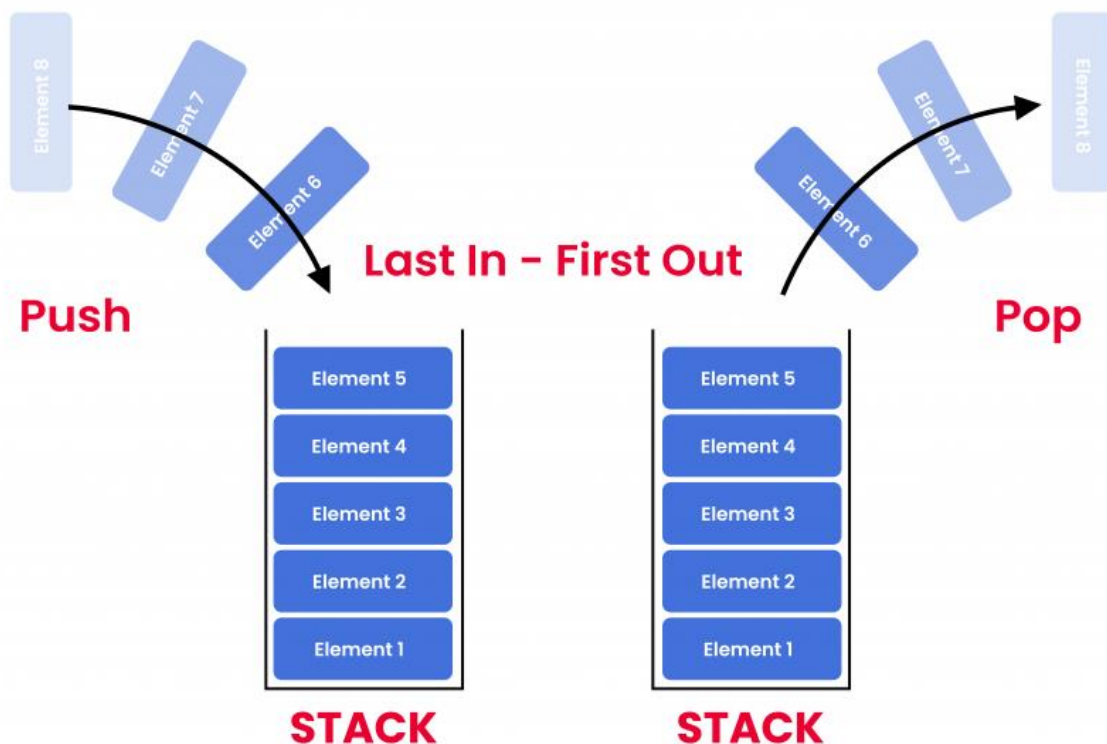


Linear - Dynamic Data Structure (Stack)

Introduction

Stack is a **linear** data structure, commonly used in most programming languages. It is named stack as it behaves like a real-world stack, for example – a deck of cards or a pile of plates, etc. At any given time, we can only access the top element of a stack. The following diagram depicts a stack and its operations:



Stack Terminology

1. **Size:** this term refers to the maximum size of the stack or the number of possibly added elements.
2. **TOP:** this term refers to the top of the stack. It is a stack pointer used to **check overflow** and **under flow** conditions. The initial value of **TOP is -1** when the stack is empty.
3. **Stack underflow:** is the situation when the stack contains no elements. At this point the top of the stack points to the **stack bottom**.
4. **Stack overflow:** is the situation when the stack is full and no more elements can be added. At this point the top of the stack points to the **highest location** in the stack.

Characteristics of a Stack

1. **Stack** is used in many different algorithms like **tree** traversal, recursion, etc.
2. **Stack** is implemented through an **array** or **linked list**.
3. It follows the **L**ast-**I**n-**F**irst-**O**ut (**LIFO**) principle, i.e., the element that is pushed at the end is popped out first.
4. The **insertion** and **deletion** are performed at **one** end from the **top** of the **stack**.
5. If the allocated space for the **stack** is full, and still anyone attempts to **add** more elements, it will lead to **stack overflow**.

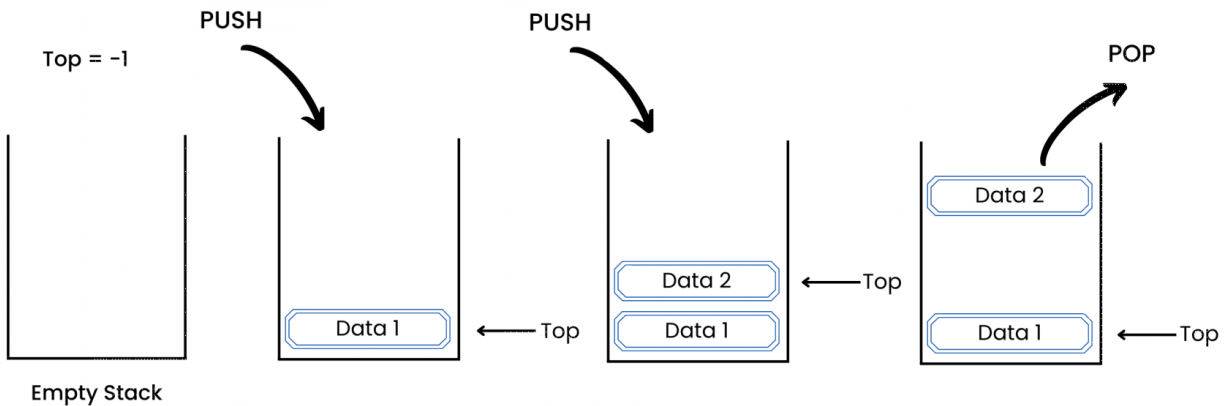
Applications of a Stack

1. The **stack** is used in the **evaluation, parenthesis** checking and **conversion** of **arithmetic expressions**.
2. While **reversing a string**, the **stack** is used as well.
3. The **stack** represents a reserved area of memory used for managing **function calls** and **local variables** during program execution.
4. The **stack** is used to perform **undo** as well as **redo** operations in **word processors**.
5. The **stack** is used in the **media players**. Useful to play the **next** and **previous** song.

Operation performed on Stack

The following are some common **operations** performed on **stacks**:

1. **Push()**: adds a data value to the top of the stack (**pushed**). There is **no way** to inserting element at any other position of the stack.
2. **Pop**: removes the last data value on top of the stack
3. **Peek**: returns the top data value of the stack, without removing it.
4. **IsEmpty**: is a function that check if the stack is **empty**. Returns **true** if stack is **empty** else returns **false**.
5. **IsFull**: is a function that returns **true** if the stack is **full**, then it is said to be an **Overflow** condition else returns **false**.
6. **Size**: is a function that returns the **number of elements** in the stack.



Algorithm of push() operation on Stack:

Push operation (putting a new data element onto stack) involves a series of steps:

Step 1 – Check if the stack is **full**.

Step 2 – If the stack **IsFull** ($\text{top} == \text{capacity} - 1$), then stack **Overflows** and produces an error message “we cannot insert the element to the stack”, then **exit**.

Step 3 – If the stack is not **IsFull**, increments **top** to **point next empty space**. ($\text{top} = \text{top} + 1$)

Step 4 – Adds new data element to the stack location where **top** is pointing.

Step 5 – The elements can be **pushed** into the stack till we reach the **capacity** of the stack.

Step 6 – Returns success.

python code for push an element in stack using array:

```
import numpy as np
memory = np.zeros(10)
print (memory)
class stack:
    top = -1
    size = 5

def push(st, val):
    if st.top >= st.size-1:
        print ("stack overflow ")
        return False
    else:
        st.top = st.top + 1
        memory[st.top] = val
        return True

st = stack
push(st, 5)
print (memory)
```

In the above program, the `push()` function takes argument `val` i.e., value to be pushed into the stack. If a `top` is greater than or equal to `size`, there is no space in a stack and `overflow` is printed. Otherwise, `val` is pushed into the stack.

Algorithm of pop operation on Stack:

Step 1 – before `popping` the element from the `stack`, we check if the stack is `empty`.

Step 2 – If the stack is `empty` (`top == -1`), then Stack **Underflows** and we cannot remove any element from the stack, produces an error message and exit

Step 3 – If the stack is not empty, accesses the data element at which `top` is pointing.

Step 4 – Otherwise, **decreases** the value of `top` by 1 (`top = top - 1`)

Step 5 – Return the stored `top` value.

python code for pop function in stack:

```
def pop(st):  
    if st.top < 0:  
        print ("stack underflow ")  
        return False  
    else:  
        val = memory[st.top]  
        memory[st.top]=0  
        st.top = st.top - 1  
        return val
```

The `pop()` function pops the `topmost` value of the stack, if there is any value. If the stack is empty then `underflow` is printed. This is given as follows: